

Content Aggregator - Project Planning Document

Overview

A scheduled content aggregation tool that retrieves emails matching criteria, summarizes them via LLM, and surfaces top “stories” in a daily digest.

Core Requirements (v0)

1. Run on a schedule (cron, daily)
2. Log into email and retrieve messages matching criteria (sender, date)
3. Store retrieved emails as markdown files locally
4. Summarize each email via LLM
5. Review summaries and surface top stories in an email body with links

Decisions Made

Email Source

- **Chosen:** Gmail API
- **Rationale:** Clean structured access, powerful search query language, OAuth2 handles auth flow, aligns with existing Google account

Output Method (v0)

- **Chosen:** CLI/stdout + local markdown files
- **Rationale:** Simpler than web server, meets v0 requirements

Storage Location

- **Chosen:** ~/bin/content_aggregator/
- **Structure:**
 - bin/ - CLI entry point
 - lib/ - Core modules
 - data/ - Stored emails, summaries, digests

Language & Conventions

- **Chosen:** Ruby, following code-skillz guidelines
- **Key conventions:**
 - Simple, readable code (avoid clever idioms)
 - Shallow control flow, early returns
 - No new abstractions unless explicitly requested
 - Descriptive variable names
 - Internal configuration over CLI args
 - 2-space indentation (Ruby standard)
 - `# frozen_string_literal: true` for production scripts

API Key Handling

- **Current:** Hardcoded in config.yml
- **Deferred:** Better secrets handling method (v1+)
- **Note:** Using your existing OpenAI key from jobs_agent.rb

Tradeoffs Considered

Gmail API vs IMAP vs MBOX

Approach	Pros	Cons
Gmail API	Clean API, powerful search, OAuth	Requires Google Cloud setup, quota limits
IMAP	Simpler auth, works with any server	App-specific password needed for Gmail
MBOX	No auth complexity, works offline	Manual export required

Decision: Start with Gmail API, MBOX could be v1 fallback

Ranking Algorithm

- Simple weighted scoring based on sender reputation, keywords, attachments, recency
- Configurable weights in config.yml
- Deferred: More sophisticated NLP-based ranking (future)

Deferred Items

1. **Secrets handling** - Currently in config.yml, need better solution
2. **IMAP adapter** - Stub only, not implemented
3. **MBOX adapter** - Stub only, not implemented
4. **Cron setup** - Example in config, not active
5. **Web interface** - CLI output only for v0
6. **Email delivery** - Digest saved to file, not emailed
7. **OAuth flow** - Need refresh token, requires Google Cloud setup

Work Accomplished

Project Structure

```
~/bin/content_aggregator/  
  bin/  
    content_aggregator          # Main CLI entry point
```

```

lib/
  email_fetcher.rb      # Gmail adapter with full message fetch
  storage.rb            # Markdown file persistence
  summarizer.rb         # LLM integration
  aggregator.rb         # Ranking algorithm
  formatter.rb          # Digest output
data/                   # Created for storage
  emails/
  summaries/
  digests/
config.yml.example      # Configuration template

```

Implemented Features

1. **EmailFetcher** - GmailAdapter
 - OAuth token refresh
 - Search query building (from, subject, after, before, unread, attachments)
 - Full message retrieval (headers, body, snippet)
 - Blocked senders filtering
2. **Storage**
 - Email markdown formatting
 - Dated directory structure (YYYY-MM-DD)
 - Filename sanitization
 - Digest storage
3. **Summarizer**
 - OpenAI API integration
 - Configurable model (gpt-4.1-2025-04-14 default)
 - Summary markdown generation
4. **Aggregator**
 - Scoring algorithm with:
 - Sender weights (configurable)
 - Keyword bonuses
 - Attachment bonus
 - Recency bonus
 - Content length bonus
 - Top N selection
5. **Formatter**
 - Markdown digest output
 - Story numbering
 - Links to original emails
6. **CLI**
 - Config file loading
 - Dry-run flag
 - Verbose flag

Configuration (TBD)

```
email:
  source: gmail
  gmail:
    refresh_token: TBD # From Google Cloud OAuth
    client_id: TBD # From Google Cloud
    client_secret: TBD # From Google Cloud
  filters:
    unread_only: true
    blocked_senders:
      - noreply@
      - newsletter@

llm:
  api_key: TBD # From OpenAI

ranking:
  top_count: 5
  sender_weights: {}
  keyword_bonus:
    urgent: 2.0
    action: 1.0
```

Next Steps

1. **Obtain Gmail credentials**
 - Set up Google Cloud project with Gmail API
 - Create OAuth credentials (Desktop app)
 - Exchange authorization code for refresh token
 - Fill in: refresh_token, client_id, client_secret
2. **Obtain OpenAI API key**
 - Get from <https://platform.openai.com/api-keys>
 - Fill in: llm.api_key
3. **Test the pipeline**
 - Run: `~/bin/content_aggregator/bin/content_aggregator -v`
 - Verify emails are fetched
 - Verify summaries are generated
 - Verify digest is created
4. **Refine filters**
 - Adjust email filters for desired content
 - Configure sender_weights for ranking
 - Tune keyword_bonus values
5. **Set up cron**
 - Uncomment schedule in config
 - Add to crontab

Future Enhancements (v1+)

- Better secrets handling (environment variables, keychain)
- IMAP/MBOX adapters
- Email digest delivery
- Web interface for browsing
- More sophisticated ranking algorithm
- Duplicate detection
- Multi-account support